

Big Data

Segunda Evaluación Parcial

Implementación técnica y MVP end-to-end mínimo
Proyecto integrador — Cloud Provider Analytics

Santiago Bolado (64261)	Felipe Zanel Suarez (64914)	Gonzalo Martinez Pagola (64289)	Bautista Manuel Camiño (62075)

Fecha de entrega: 16 de noviembre de 2026

Trazabilidad con la consigna

§	Sección	Componente de la consigna (§6.2)
1	Resumen ejecutivo	—
2	Respuesta al feedback y correcciones	Correcciones y arquitectura
3	Arquitectura implementada	Correcciones y arquitectura
4	Pipeline por zona	Batch a Bronze · Streaming a Bronze · Silver · Gold
5	Calidad y quarantine	Calidad
6	Serving en AstraDB	Serving
7	Componente analítico	Analítica o ML
8	Idempotencia, pruebas y reproducibilidad	Idempotencia · Pruebas y reproducibilidad
9	Gobierno preliminar	Gobierno preliminar
10	Decisiones, limitaciones y backlog	Backlog final
A–E	Anexos (documento aparte)	Evidencias de ejecución (§6.3)

1. Resumen ejecutivo

Qué se entrega. Un pipeline ejecutable que lleva los datos de Landing a Bronze, Silver, Gold y AstraDB, con ingesta batch de las siete fuentes, Structured Streaming de los 120 archivos de eventos, el mart FinOps `org_daily_usage_by_service`, un componente de detección de anomalías y dos consultas servidas desde Cassandra. Corre completo con un comando (`python -m src.pipeline --reset`) o desde el notebook de Colab, y se verificó dos veces desde un entorno limpio con resultados idénticos.

Resultados.

Indicador	Resultado
Eventos procesados	43.200 de 43.200 en Bronze y en Silver; ninguno descartado por una regla de calidad
Mart FinOps (Gold)	11.050 filas (80 organizaciones × 60 días × 6 servicios); USD 147.433,98, cuadra con Silver
Late data	36.808 eventos tardíos (85,2%) fuera de la capa rápida, recuperados por la reconciliación
Serving	AstraDB: 11.050 + 12.314 filas; Q1 en 39 ms y Q2 en 38 ms
Anomalías	130 en 11.050 celdas; detecta los 54 picos inyectados en el dataset
Idempotencia	Pipeline y carga a AstraDB ejecutados dos veces: variación 0 en 12 tablas
Pruebas	47 tests automáticos; gobierno con control que falla si una columna no está documentada

Lo que confirmó el dataset completo. El TP1 dejó abierta la pregunta de si los archivos de eventos llegan ordenados (A-07), porque se había perfilado uno solo. Son 120, y cada uno cubre 59 a 60 días sin orden temporal. Con el watermark de 7 días que fijó el diseño, la ruta streaming deja afuera el 85,2% de los eventos y cubre solo el 15,1% del costo. La reconciliación batch del patrón híbrido no era una precaución: es lo que hace correcto al mart.

Lo que encontró la implementación. Las pruebas detectaron cuatro problemas que el diseño no anticipaba: en Spark 4 un timestamp inválido detiene la consulta de streaming en vez de devolver nulo; la métrica de Spark para tardíos los subcuenta; deduplicar con un watermark sobre la fecha del evento habría hecho perder el 87,6% de Bronze; y la primera versión de la deduplicación batch podía conservar la copia y aislar el original. Los cuatro están corregidos y cubiertos por un test (§4, §8).

Estado frente a los objetivos del TP1.

#	Objetivo	Resultado	Estado
O1	Frescura ≤ 5 min	Latencia por micro-lote: mediana 1,3 s, máximo 12,7 s	Cumple
O2	Completitud batch	7 de 7 fuentes conformadas en Bronze	Cumple
O3	Inválidos aislados con motivo	Quarantine por regla; nada se descarta en silencio	Cumple
O4	Reprocesar sin duplicados	Variación 0 en 12 tablas	Cumple
O5	Cobertura P1 a P5	P1 y P2 servidas; P3 a P5 en el backlog	Parcial (final)
O6	Consulta < 1 s	39 ms y 38 ms	Cumple
O7	Reproducible desde cero	Colab y entorno local dan cifras idénticas	Cumple
O8	v1 y v2 en el mismo mart	Esquema ampliado con nulos explícitos; test	Cumple

---

2. Respuesta al feedback y correcciones

La única dimensión por debajo de 10 en la primera entrega fue Ingeniería y documentación: el documento llegó a 41 páginas. La indicación fue mantener la profundidad y llevar los detalles secundarios a anexos. Este documento afirma cada resultado y remite a su evidencia: el detalle está en el documento de Anexos y en el repositorio (`DECISIONS.md`, `evidence/`, `docs/`). La estructura y el presupuesto de páginas se fijaron antes de escribir.

Plan de correcciones (`docs/plan_correcciones.md`):

#	Corrección	Estado
C-01	Documento principal de 10 a 12 páginas, detalle en anexos	Este documento
C-02	Duplicados 17/61/9 del TP1: salían de un conteo aproximado; el exacto da 0	Corregido (D-011)
C-03	Maestros sin particionar en D-005; la consigna pide Parquet particionado	Corregido (D-011)
C-04	Cerrar A-07 con el dataset completo	Cerrada (D-012)
C-05	Diagrama que represente lo implementado	Diagrama v2 (§3)
C-06	Cifras de volumen estimadas con ~20 archivos	Corregidas (tabla siguiente)

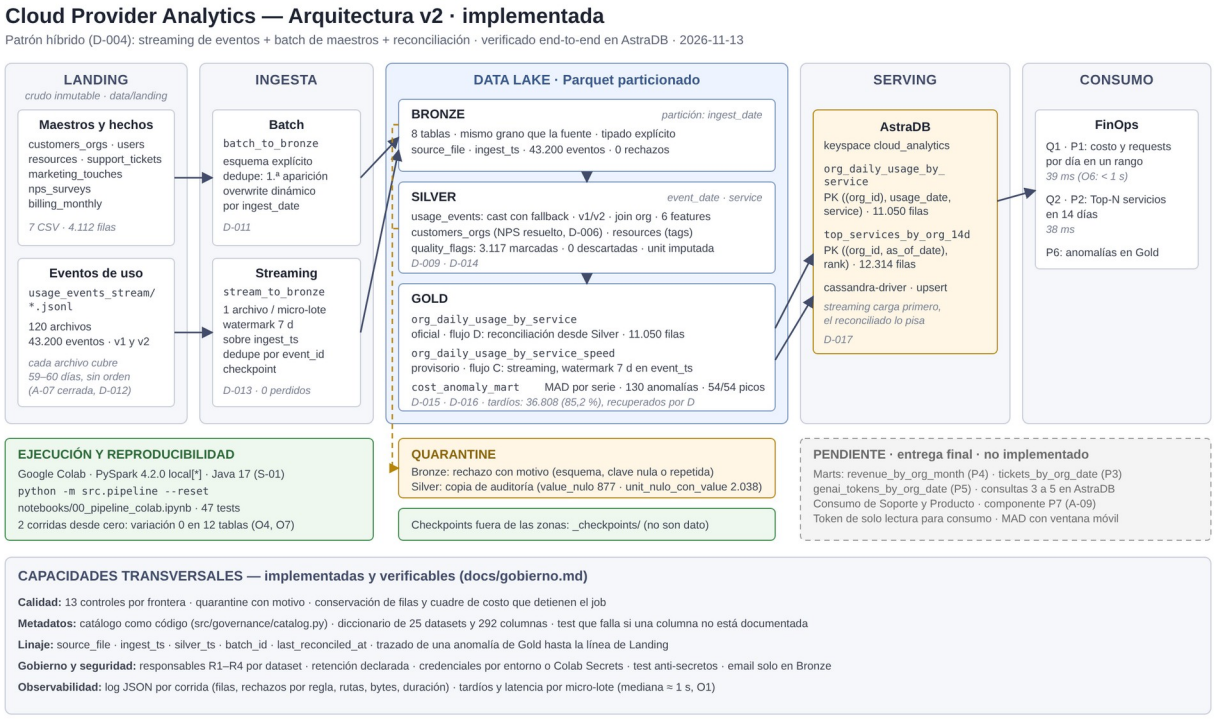
Cifras del TP1 corregidas con el dataset completo:

Cifra	TP1	Real
Tamaño de Landing	2,46 MB	13,2 MB (0,3 MB de CSV + 12,9 MB de eventos)
Archivos y eventos de uso	~20 · ~7.200	120 · 43.200
Eventos por día	~120	720
Eventos por recurso-hora	0,0125	0,075
Claves repetidas (tickets, marketing, billing)	17 · 61 · 9	0 · 0 · 0
Eventos dentro de un watermark de 7 días	8,9% (1 archivo)	12,4% simulado · 14,8% medido

Ninguna corrección cambia una conclusión del TP1: el volumen sigue sin justificar Big Data por sí solo, y el desorden de los eventos resultó aún más marcado de lo estimado.

---

3. Arquitectura implementada



Arquitectura v2 (13/11/2026). Solo componentes que corren; lo previsto para la entrega final va en el recuadro punteado. Fuente: docs/diagrama_arquitectura_v2.svg

El patrón es el híbrido de D-004 y ahora está validado con datos: rutas segmentadas por fuente (streaming para eventos, batch para maestros) y una reconciliación batch que corrige lo que el watermark descarta. Los cambios respecto de la v1 que importan para la defensa:

- **Landing son las fuentes.** La v1 mostraba a la ingesta escribiendo en Landing, que es inmutable; la ingesta solo lee de ahí.
- **Dos watermarks, no uno.** Uno sobre la hora de llegada para deduplicar en Bronze sin perder eventos, y otro sobre la hora del evento para la agregación del mart (D-013).
- **La capa rápida escribe en una tabla propia.** Parquet no tiene upsert: la fusión entre la ruta streaming y la reconciliación ocurre en Cassandra (D-015).
- **El modelo de serving cambió dos claves** respecto del TP1, por las razones de la §6 (D-017).

---

4. Pipeline por zona

4.1 Ingesta batch a Bronze

`batch_to_bronze` lee las siete fuentes CSV con esquema explícito (nunca `inferSchema`) y escribe Bronze en Parquet, particionado por `ingest_date`, con `ingest_ts` y `source_file`. Tres decisiones (D-011):

- **Una fila que no respeta el esquema no se nulifica en silencio.** Spark la guarda en `_corrupt_record` y va a quarantine con la línea cruda.

- **Re-ejecutar no duplica ni borra historia.** `overwrite` con `partitionOverwriteMode=dynamic` reemplaza solo la partición del día. `append` duplicaba; el overwrite total borraba las fotos anteriores.
- **Ante una clave repetida se conserva la primera aparición** en el archivo y las siguientes van a quarantine (`clave_duplicada`). El dataset no tiene ninguna, pero la regla garantiza la idempotencia ante una re-entrega.

Resultado: 4.112 filas en 7 tablas, 0 rechazos, un archivo por partición. Los nulos y las violaciones de invariantes en Bronze coinciden uno a uno con el perfilado del TP1: Bronze tipa, pero no limpia.

4.2 Streaming a Bronze y cierre de A-07

Antes del streaming se perfilaron los 120 archivos en batch (`notebooks/02_perfilado_eventos_a07.py`). Cada uno cubre 59 a 60 días y la correlación entre el número de archivo y la fecha de sus eventos es −0,015. Procesando un archivo por micro-lote, el porcentaje de eventos que un watermark sobre la fecha del evento deja afuera es:

Watermark	7 días	30 días	60 días
Eventos descartados	87,6%	49,6%	0%

Esto expone un problema que el diseño no veía. La deduplicación con watermark de Structured Streaming descarta sin dejar rastro todo lo que queda detrás del umbral: aplicada al escribir Bronze con 7 días sobre `event_ts`, Bronze perdía el 87,6% y la reconciliación se quedaba sin insumo. **Decisión (D-013): dos watermarks.** Bronze deduplica con `withWatermark("ingest_ts", "7 days").dropDuplicatesWithinWatermark(["event_id"])`: la hora de llegada siempre avanza, así que ningún evento queda atrás, y se descartan solo las re-entregas del mismo `event_id`. El watermark de 7 días sobre `event_ts` (D-008) se aplica en la agregación del mart, donde descartar es aceptable porque Bronze conserva todo.

Resultado: 120 micro-lotes de 360 eventos (`maxFilesPerTrigger = 1`), 43.200 eventos en Bronze, 0 descartados. Re-ejecutar con el mismo checkpoint procesa 0 filas; re-entregar un archivo con otro nombre se lee y no se escribe. Un JSON roto, un `event_id` nulo o un timestamp inválido van a quarantine. La prueba de este último caso mostró que en Spark 4 `to_timestamp` sobre un valor inválido detiene toda la consulta; se usa `try_to_timestamp`.

4.3 Silver

`silver_job` recomputa Silver desde Bronze. Las funciones de conformación son puras y las usa también la ruta streaming, con lo que las dos rutas no pueden divergir (§6.7 del TP1).

- **Eventos** particionados por `event_date` y `service` (D-005): `value` casteado con fallback (D-003), v1 promovido al esquema ampliado con `carbon_kg` nulo (no medido, distinto de 0), join con la organización y deduplicación por `event_id` sobre la tabla completa.
- **Seis features:** `requests`, `cpu_hours`, `storage_gb_hours`, `cost_usd`, `genai_tokens` y `carbon_kg`. El formato largo `metric`/`value` se proyecta a columnas en Silver para que Gold solo sume.
- **Maestros:** `customers_orgs` con el NPS resuelto según D-006 (52 organizaciones por encuesta, 25 por CRM, 3 sin dato) y `resources` con `tags_json` convertido a array.

Resultado: 43.200 eventos de 43.200, en 360 particiones con un archivo cada una. Las reglas de calidad de esta frontera están en la §5.

4.4 Gold: el mart FinOps por dos rutas

`org_daily_usage_by_service` (grano: organización, día, servicio) se calcula con una única lista de agregaciones, todas sumas o conteos, que comparten las dos rutas (D-015):

	Flujo C · capa rápida	**Flujo D · reconciliación**
Entrada	Bronze como stream, un archivo por micro-lote	Silver completo
Watermark	7 días sobre `event_ts`, ventana diaria	No aplica
Salida	`…_speed`, particionado por `batch_id`	Mart oficial, por `usage_date`, con `last_reconciled_at`
Eventos incorporados	6.392 (14,8%)	43.200 (100%)
Costo cubierto	USD 22.219 (15,1%)	USD 147.433,98 (100%)
Idempotencia	Checkpoint; `foreachBatch` reescribe su `batch_id`	Recomputa y sobrescribe

Dos hallazgos para la defensa. La métrica `numRowsDroppedByWatermark` de Spark no sirve para medir tardíos, porque se filtran antes del operador de estado y los subcuenta. Se miden de forma exacta: eventos en Bronze menos eventos incorporados a la capa rápida, que da 36.808. Además, Spark 4 filtra con el watermark del lote anterior, así que los descartes empiezan en el tercer micro-lote. Con un solo micro-lote las dos rutas producen exactamente el mismo mart, lo que verifica un test (riesgo G-04 del TP1).

---

5. Calidad y quarantine

Hay dos clases de controles: los que aíslan o marcan registros, y los que detienen el job si algo no cuadra. Quarantine significa cosas distintas según la frontera: en Bronze, lo que va a quarantine no entra; en Silver es una copia de auditoría y el evento sigue con su flag, porque el rechazo opera sobre el campo derivado y no sobre el registro (D-009).

Frontera	**Regla**	**Acción**	**Casos**
Landing → Bronze	Fila no parseable · clave nula o repetida · `event_id` nulo · timestamp inválido	Quarantine con motivo	0
Bronze → Silver	`value` nulo	Métrica nula; el costo promueve; copia	877
Bronze → Silver	`unit` nulo con `value` presente	Unidad imputada desde la métrica; flag; copia	2.038
Bronze → Silver	`cost_usd_increment` < −0,01	Flag `is_credit_adjustment`	211
Bronze → Silver	`genai_tokens` ⇔ v2 y genai · org sin dimensión · duplicado	Flag y copia	0
Cada job	Conservación de filas · grano único · cuadre de costo con Silver	Detiene el job	Se cumple

La imputación de `unit` es una decisión nueva (D-014): en los 41.125 eventos que traen unidad, la unidad es función de la métrica sin excepciones (`cpu_hours → hours`). Se corrige y se marca, con el mismo criterio que la tasa de cambio en USD de D-007. Gracias a D-009, los 877 eventos sin valor de métrica conservan USD 2.968 de costo que llegan al mart. Muestras de cada regla: Anexo B.

---

6. Serving en AstraDB

El modelo es query-first: la tabla se diseña desde la consulta. Un mismo mart origina dos tablas porque P1 y P2 necesitan órdenes distintos. Esto cierra A-08 (D-017).

Tabla	Clave primaria	Responde	Filas
`org_daily_usage_by_service`	((org_id), usage_date, service)	P1: costo y requests por día en un rango	11.050
`top_services_by_org_14d`	((org_id, as_of_date), rank)	P2: Top-N servicios en 14 días	12.314

Dos claves cambiaron respecto del TP1. En P1, la fecha pasó de la partición a la clustering key: con la fecha en la partición, un rango obliga a una consulta por día; como clustering, el rango es un corte contiguo de una sola partición. En P2, el orden pasó del costo al `rank`: con el costo en la clave, recalcularlo crea una fila nueva y deja la vieja, y el upsert deja de ser idempotente. El Top-N se precalcula en Spark para cada fecha con 14 días completos, porque Cassandra no agrega ni ordena por columnas calculadas.

La carga usa `cassandra-driver` en `foreachPartition`, con sentencias preparadas. La consigna admite conector o driver, y el Spark Cassandra Connector es un JAR atado a versiones de Spark y Scala, sin compatibilidad asegurada con Spark 4.2. Primero se carga la capa rápida y después el reconciliado la pisa por clave primaria: es la fusión de las dos rutas.

Consulta	Resultado	Latencia (O6: < 1 s)
Q1 · `org_okep7y6w`, 01 al 07/08	25 filas; coinciden con Gold	39 ms
Q2 · `org_okep7y6w`, corte 31/08	genai 211,03 · storage 117,87 · database 90,88	38 ms

Una segunda carga reenvía las 23.364 filas y el `COUNT(*)` no cambia. CQL, salidas y capturas de la consola de AstraDB: Anexo C.

---

7. Componente analítico: anomalías de costo

`cost_anomaly_mart` aplica MAD sobre el costo diario bruto de cada serie (organización, servicio) del mart reconciliado, con umbral |score| > 3,5 (D-010, D-016). Cada día se compara con el comportamiento habitual de esa misma organización en ese servicio. Los créditos se marcan aparte y no compiten con la detección de picos. Las 262 series tienen entre 24 y 60 días, por encima del mínimo de 20; el fallback a la estadística del servicio queda implementado y probado.

No hay etiquetas, pero el dataset inyecta picos: eventos de más de USD 50 con una mediana de USD 1. Sobre el mismo mart:

Método	Celdas marcadas	Picos detectados (de 54)
z-score global, |z| > 3	93	51
MAD global, |score| > 3,5	405	54
MAD por serie (elegido)	**130 (1,18%)**	**54**

El z-score global pierde picos en servicios baratos; el MAD global los encuentra, pero marca tres veces más celdas porque confunde caro con anómalo. **Límites:** sin etiquetas no se puede medir precisión, y las 76 celdas marcadas que no contienen un pico inyectado pueden ser anomalías reales o falsos positivos. La

línea de base usa la serie completa; una ventana móvil queda en el backlog. Este componente es el que cubre Analítica o ML en esta entrega; el alcance final se decide en A-09.

---

8. Idempotencia, pruebas y reproducibilidad

Idempotencia (O4). El pipeline completo y la carga a AstraDB se ejecutaron dos veces desde un reinicio limpio, en Colab. Las 12 tablas dan los mismos conteos (Anexo A):

Capa	Mecanismo	Corrida 1 = Corrida 2
Bronze batch	Overwrite dinámico por `ingest_date`	7 tablas, 4.112 filas
Bronze streaming	Checkpoint + dedupe por `event_id`	43.200 (2.ª: 0 lotes)
Silver y Gold	Recomputación con overwrite	43.200 · 11.050
AstraDB	Upsert por clave primaria	11.050 · 12.314

Pruebas. 47 tests con pytest cubren cada frontera: tipado, columnas técnicas, quarantine con filas sucias inyectadas, re-entrega, compatibilidad v1/v2, regla compuesta de GenAI, tardíos, equivalencia entre rutas, casos borde de MAD, forma de las tablas de serving, documentación de cada columna y ausencia de credenciales. Las corridas que encontraron los cuatro bugs de la §1 quedaron como tests de regresión.

Reproducibilidad (O7). El notebook `00_pipeline_colab.ipynb` instala Java 17, clona el repo y corre todo desde cero. Colab y el entorno de desarrollo dan cifras idénticas. Configuración externalizada en variables de entorno (`config/.env.example`). Manejo de errores: cada job valida conservación y cuadre y termina con código distinto de cero si no se cumplen; las credenciales faltantes fallan antes de conectar, con un mensaje que indica qué definir.

---

9. Gobierno preliminar

- **Responsables.** Roles R1 a R4 del TP1, con un dueño por cada uno de los 25 datasets.
- **Metadatos como código.** `src/governance/catalog.py` describe cada dataset y cada una de sus 292 columnas; el diccionario de datos se genera desde ahí. Un test falla si el pipeline produce una columna sin descripción: se verificó borrando una.
- **Linaje.** Además de `source_file`, `ingest_ts`, `batch_id` y `last_reconciled_at`, un trazado real sigue la anomalía de USD 191,90 del 07/08 desde Gold hasta la línea 48 de `events_part_0109.jsonl`. Sus tres eventos llegaron en tres archivos distintos: la fragmentación de A-07 en un caso concreto.
- **Acceso.** Credenciales solo por variables de entorno o Colab Secrets, y un test que busca tokens y claves en el repo. El único dato personal (`users.email`) no pasa de Bronze. La carga usa un token de administrador, porque crea tablas; el consumo debería usar uno de solo lectura (backlog).

Detalle: `docs/gobierno.md` y `docs/diccionario_datos.md`.

---

10. Decisiones, limitaciones y backlog

Decisiones de esta entrega (detalle y alternativas descartadas en `DECISIONS.md`):

#	Decisión
D-011	Maestros particionados por `ingest_date`, overwrite por día; dedupe por primera aparición
D-012	Cierra A-07: los eventos no tienen orden temporal; se confirman D-004 y D-008
D-013	Dos watermarks: `ingest_ts` para dedupe en Bronze, `event_ts` para la agregación
D-014	`unit` nulo se imputa desde la métrica; la métrica nula no se imputa
D-015	Gold por dos rutas con una agregación compartida; tardíos medidos de forma exacta
D-016	MAD por serie sobre la historia completa; validado contra los picos inyectados
D-017	Cierra A-08: dos tablas query-first y carga con el driver de Python

Queda abierta A-09 (alcance del componente analítico, tras esta entrega).

Limitaciones declaradas.

- El entorno es Colab `local[*]`: el paralelismo es simulado y la performance a escala no se puede verificar.
- La retención por zona está definida pero no se puede ejercitar con 4 meses de datos.
- El serving responde P1 y P2; P3 a P5 corresponden a la entrega final.
- El token de AstraDB del pipeline tiene permisos de administrador.

Backlog (`docs/backlog_final.md`, Anexo E): 13 ítems obligatorios (80 horas-persona: los marts de revenue, tickets y GenAI, las consultas 3 a 5, las invariantes entre columnas del TP1, presentación, video y defensa), 6 deseables (25 h) y 7 fuera de alcance. Regla de congelamiento: nada deseable después del 30/11.

Anexos

En el documento aparte *Anexos — Segunda entrega*:

	Anexo	Fuente en el repositorio
A	Evidencias de ejecución: conteos, rutas, tamaños, tiempos e idempotencia	`evidence/runs/`, `evidence/idempotencia.md`
B	Muestras de quarantine por regla	`evidence/quarantine_muestras.md`
C	Serving: CQL, resultados y capturas de AstraDB	`infra/cql/`, `evidence/serving/`
D	Diccionario de datos (resumen) y trazado de linaje	`docs/diccionario_datos.md`, `evidence/linaje_trazado.md`
E	Backlog final	`docs/backlog_final.md`

Repositorio: `github.com/santiagobolado/tp1-cloud-provider-analytics` · ejecución: `python -m src.pipeline --reset` o `notebooks/00_pipeline_colab.ipynb`.